PSE-Net Plain: A Reference Implementation and Network Transmission Protocol for the Weighted Pair-Symbol Encoding System

Companion paper to: "A Weighted Pair-Symbol Encoding System with Relational States: From the 2-Bit Table to the 4-Bit Table"

Mohammed Khalid Almodiny — Independent Researcher · October 31, 2026

Abstract

This paper describes pse_net_plain.py, a reference implementation that takes the weighted pair-symbol encoding system from the companion paper and carries it from input to output across a network. It introduces one change to the encoding table, a within-state ladder rule for repeated values, which keeps every 3-digit section below 100 so that its leading zero can be dropped in transmission. Each 4-byte block is then sent as 12 BCD digits (48 bits). The paper specifies the frame format, the integrity check (CRC-32), the TCP exchange, the decoding procedure, and the error handling, and reports measured results: exhaustive round trips for all 1-byte and 2-byte inputs, detection of 2,000 out of 2,000 random single-bit corruptions, and identical files after transfer between separate processes. Transmission in the system's digit form costs 1.5 times the original data, and a raw mode is provided that sends the original pairs at 1.0 times while the receiver rebuilds the system's values locally. The implementation uses only the Python standard library and has no encryption; an encrypted variant is noted as separate work.

1. Relation to the Companion Paper

The companion paper defines the weights (00→2, 01→3, 10→5, 11→13), the three group types, the ten relational states, the comparison rule, and the value X.YYY of each byte. It adopts a repetition-offset table (+0.050 / +0.080 / +0.100) decoded by lookup. This implementation keeps every rule of that paper except the way repeated values are separated, for one practical reason: with the offset table, 56 of the 256 values have a non-zero first digit in YYY (for example 1.102), so that digit cannot be omitted. The ladder rule of Section 2 removes this, and is proposed for the next version of the companion paper.

Element	Companion paper	This implementation
Weights, group types, states, comparison	R1-R6	Unchanged
Base value	X + (s1 + s2)/1000	Unchanged
Repeated values	Offsets +0.050, +0.080, +0.100	Ladder within each state, step 0.001
Value range of YYY	008-123	008-061 (always below 100)
Decoding	Lookup table built by rules	Lookup table built by rules
Transmission	Not specified	12 BCD digits per 4 bytes, framed, CRC-32

2. The Within-State Ladder Rule

For each state X separately, the cells are grouped by their sum s1 + s2. The rule is applied as follows:

1. Sort the distinct sums of state X in ascending order.
2. In each repeated sum, sort the cells by the sum of the smaller group. The first cell keeps the sum.
3. Every other cell moves to the next free value above the largest sum of state X, one step (0.001) at a time, skipping values already used.
4. Store the result as the final value X.0YY of that cell.

Because each state has at most 45 cells and its largest sum is at most 52, the ladder never passes 0.099. The measured maxima per state are:

State X	0	1	2	3	4	5	6	7	8	9
Cells	10	45	45	25	35	35	25	15	15	6
Distinct sums	10	28	28	19	23	23	19	13	13	6
Largest YY after ladder	52	61	61	50	48	48	50	36	36	36

Result: 256 distinct values out of 256, largest value 9.036, and the first digit of every YYY is 0. The four example bytes of the companion paper become:

Byte	Companion paper (offset table)	This implementation (ladder)
11101000	8.025	8.025
11111000	6.083	6.033
11101011	4.036	4.036
10011010	3.018	3.018

3. Encoding Pipeline

```
input file (bytes)
  -> split into 4-byte blocks (last block padded with zero bytes)
  -> each byte -> two 4-bit groups -> four 2-bit pairs -> weights
  -> state X, sum, ladder table -> value X.0YY
  -> block number  X4 X3 X2 X1 . 0YY1 0YY2 0YY3 0YY4
  -> drop the four fixed zeros -> 12 digits  X4 X3 X2 X1 YY1 YY2 YY3 YY4
  -> BCD: 4 bits per digit -> 48 bits (6 bytes) per block
  -> frame: header + payload
```

Worked example. Bytes e8 f8 eb 9a (11101000 11111000 11101011 10011010):

Step	Result
Block number	3468.025033036018
Digits sent (zeros dropped)	346825333618
BCD bits (48)	0011 0100 0110 1000 0010 0101 0011 0011 0011 0110 0001 1000
Receiver restores zeros	3468.025033036018
Lookup per section	8.025 → 11101000, 6.033 → 11111000, 4.036 → 11101011, 3.018 → 10011010

4. Frame Format

Every message is one frame. All multi-byte fields are big-endian, so the format is identical on Windows, Linux, and macOS.

Field	Size (bytes)	Value
Magic	4	"PSE2"
Mode	1	1 = bcd (system digits), 2 = raw (original bytes)
Flags	1	0 (bit 0 reserved for encryption; refused by this version)
Original length	4	Number of data bytes before padding
Body length	4	Number of payload bytes that follow
CRC-32	4	zlib CRC-32 of the original data
Payload	variable	bcd: 6 bytes per 4-byte block; raw: the data itself

The header is 18 bytes. In bcd mode each 4-bit nibble holds one digit (0-9); the nibble value F is reserved as padding and never appears in valid digits.

5. Network Exchange (TCP)

1. The sender opens a TCP connection to the receiver.
2. It sends a 4-byte big-endian length, followed by the frame.

3. The receiver reads exactly that many bytes, parses and verifies the frame (Section 6).
4. It replies with 2 bytes: "OK" if the data was decoded and the CRC-32 matched, or "ER" otherwise.
5. Both sides close the connection. The receiver continues listening unless started with --once.

Frames larger than 2 GB are refused before reading. A timeout of 30 seconds applies to each connection, and the listening socket polls every second so that the receiver can be stopped with Ctrl+C on both Windows and Linux.

6. Decoding and Verification

1. Check that the frame is at least 18 bytes and starts with "PSE2"; check that the body is complete.
2. Refuse the frame if the encryption flag is set.
3. bcd mode: unpack the nibbles into digits, split into 12-digit blocks, restore the four zeros, rebuild X.0YY for each section, and look each value up in the ladder table.
4. raw mode: take the payload as the data, and rebuild the X.0YY numbers locally for analysis.
5. Trim the padding using the original length.
6. Recompute CRC-32 and compare with the header.

Error condition	Response
Frame shorter than the header, wrong magic, truncated body	Rejected with a message; receiver keeps running
Encrypted frame	Rejected: use the secure variant
Invalid BCD digit or value not in the table	Rejected
CRC-32 mismatch	Rejected as corrupted in transit
Connection dropped or empty connection	Logged; receiver keeps running

7. Measured Results

All results were measured on Linux with Python 3.12.3, using the built-in self-test and separate sender and receiver processes on the same machine (127.0.0.1).

Test	Result
Ladder table	256 distinct values; largest 9.036; every section starts with 0
All 256 one-byte inputs, both modes	All decoded exactly
All 65,536 two-byte inputs (bcd)	All decoded exactly
Inputs of 0, 1, 3, 4, 5, 1,000 bytes	All decoded exactly (padding handled)
5 MB file	Encoding 7.1 s, decoding 7.5 s
2,000 random single-bit flips in the frame	2,000 detected
Two bytes swapped (raw mode)	Detected by CRC-32
Malformed frames (short, wrong magic)	Rejected
TCP transfer, separate processes	Received file identical to the original
Malformed and empty connections, then a valid transfer	"ER" returned, receiver kept running, valid file identical

Transmission size

Data size	bcd frame	raw frame	bcd / data
4 bytes	24 bytes	22 bytes	6.0
1 KB	1,518 bytes	1,018 bytes	1.52
100 KB	150,018 bytes	100,018 bytes	1.50
1 MB	1,500,018 bytes	1,000,018 bytes	1.50

Each frame also travels with the 4-byte TCP length prefix. For small messages the fixed 18-byte header dominates; from about 1 KB upward the bcd payload converges to 1.5 times the data.

8. Usage

```
python pse_net_plain.py selftest
python pse_net_plain.py receive --port 5050 --out received.bin [--once]
python pse_net_plain.py send --host 192.168.1.10 --port 5050 file.bin [--mode bcd|raw]
python pse_net_plain.py encode file.bin
python pse_net_plain.py decode 3468.025033036018
```

Requirements: Python 3.6 or later; no third-party packages. On first use, the operating system firewall may ask to allow the listening port.

9. Constraints

The implementation inherits the constraints of the companion paper and adds the following:

- Size: the bcd mode sends 1.5 times the original data. The raw mode sends 1.0 times; no mode is smaller than the data, because the encoding is a representation, not a compressor.
- Fixed overhead: 18 bytes of header (plus 4 bytes of TCP length) per message, so very small messages are inefficient; data should be batched into larger frames.
- Memory: the whole file is held in memory on both sides; the length fields limit a frame to 4 GB, and frames above 2 GB are refused.
- Speed: about 1.5 s per MB in pure Python; a compiled implementation would be needed for high throughput.
- Concurrency: the receiver serves one connection at a time.
- Security: no encryption and no authentication. CRC-32 detects accidental corruption, not deliberate tampering. Data that must be protected should use the encrypted variant (AES-256-GCM), described separately.
- Compatibility: frames of this version (PSE2) are not compatible with earlier prototypes (PSE1), and encrypted frames are refused.
- Testing scope: tests were run on Linux on a single machine. Windows and two-machine transfers are expected to work, since only standard, platform-independent modules and big-endian fields are used, but they were not measured.

10. Conclusion

PSE-Net Plain shows that the weighted pair-symbol system can be used end to end: data is encoded into the system's numbers, transmitted, verified, and decoded back to identical bytes. The within-state ladder rule keeps every section below 100, which lets the transmission drop four digits per block and reach 12 digits (48 bits) per 4 bytes. The measured overhead (1.5 times in bcd mode) is the cost of carrying the system's description of the data; the raw mode shows that the same numbers can be rebuilt at the receiver without that cost on the wire. The next steps are to update the companion paper to the ladder rule, to measure the implementation on Windows and across two machines, and to document the encrypted variant.

References

[1] Almodiny, M. K. A Weighted Pair-Symbol Encoding System with Relational States: From the 2-Bit Table to the 4-Bit Table. 2026.

[2] Postel, J. (1981). Transmission Control Protocol. RFC 793, IETF. (Updated by RFC 9293, 2022.)

[3] Deutsch, P., & Gailly, J.-L. (1996). ZLIB Compressed Data Format Specification version 3.3. RFC 1950, IETF.

[4] IEEE Computer Society. (2019). IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019).

[5] Cowlishaw, M. F. (2002). Densely packed decimal encoding. IEE Proceedings - Computers and Digital Techniques, 149(3), 102-104.

Appendix A. Source Code: pse_net_plain.py

The complete source of the reference implementation (Python 3.6+, standard library only). Line numbers are added for reference and are not part of the file.

```
#!/usr/bin/env python3
"""
pse_net_plain.py — Weighted Pair-Symbol Encoding (no encryption): input -> encoding -> network -> decoding -> output

Pipeline
    bytes (8 bits)  ->  two 4-bit groups -> four 2-bit pairs (weights 00:2 01:3 10:5 11:13)
                    ->  value X.0YY  (X = relational state 0-9, YY = sum, ladder for repeats)
    4 bytes         ->  number  X4 X3 X2 X1 . 0YY1 0YY2 0YY3 0YY4
    transmission    ->  the fixed zeros are dropped: 12 BCD digits per 4 bytes (48 bits)
    frame           ->  header (magic, mode, flags, lengths, CRC-32) + payload

Modes
    bcd  : sends the system's digits (12 bits per byte)       -- the paper's representation
    raw  : sends the original pairs (8 bits per byte)         -- smallest possible transmission;
           the receiver still rebuilds the X.0YY numbers locally for analysis/verification

Commands
    python3 pse_net_plain.py encode  <file>                     print the numbers for a file
    python3 pse_net_plain.py decode  <num> [<num> ...]          numbers -> bytes (hex)
    python3 pse_net_plain.py receive --port 5050 --out recv.bin [--once]
    python3 pse_net_plain.py send    --host 127.0.0.1 --port 5050 <file> [--mode bcd|raw]
    python3 pse_net_plain.py selftest                            table, codec and network loopback tests

Integrity: CRC-32 (zlib), which also detects swapped bytes.
No encryption: use pse_net_secure.py for AES-256-GCM.

Only the Python standard library is used.
"""
import argparse
import os
import socket
import struct
import sys
import threading
import zlib
from collections import defaultdict

# ================================================================== rules
WEIGHTS = {"00": 2, "01": 3, "10": 5, "11": 13}            # R2
GROUPS = [f"{i:04b}" for i in range(16)]
STATE_VALUE = {1: 1, 2: 2, 3: 3, 4: 4, 5: 5, 6: 6, 7: 7, 8: 8, 9: 9, 10: 0}


def gsum(g):                      # R4
    return WEIGHTS[g[:2]] + WEIGHTS[g[2:]]


def glarger(g):                   # R3: 1 = starts larger, 0 = starts smaller or equal
    return 1 if WEIGHTS[g[:2]] > WEIGHTS[g[2:]] else 0


def first_smaller(g1, g2):        # R5
    s1, s2 = gsum(g1), gsum(g2)
    return s1 < s2 if s1 != s2 else glarger(g1) < glarger(g2)


def state(g1, g2):                # R6: ten relational states
    f1, f2 = glarger(g1), glarger(g2)
    if g1 == g2:
```

```
        return 9 if f1 else 10
    base = {(0, 0): 1, (1, 0): 3, (0, 1): 5, (1, 1): 7}[(f1, f2)]
    return base if first_smaller(g1, g2) else base + 1


def build_table():
    """Return (enc, dec): byte (0-255) <-> value in units of 0.001.

    Repetitions are resolved with the ladder rule inside each state:
    the first cell of a repeated sum keeps it (smallest smaller-group sum first),
    the others move one step (0.001) at a time above the largest sum of that state.
    All values stay below X.100, so the first digit of every 3-digit section is 0.
    """
    by_state = defaultdict(lambda: defaultdict(list))
    for g1 in GROUPS:
        for g2 in GROUPS:
            x = STATE_VALUE[state(g1, g2)]
            s = gsum(g1) + gsum(g2)
            by_state[x][s].append((min(gsum(g1), gsum(g2)), g1 + g2))
    enc = {}
    for x, sums in by_state.items():
        used = set(sums)
        nxt = max(used)
        for s in sorted(sums):
            cells = sorted(sums[s])
            enc[int(cells[0][1], 2)] = x * 1000 + s
            for _, bits in cells[1:]:
                nxt += 1
                while nxt in used:
                    nxt += 1
                used.add(nxt)
                enc[int(bits, 2)] = x * 1000 + nxt
    dec = {v: b for b, v in enc.items()}
    assert len(enc) == 256 and len(dec) == 256, "table is not one-to-one"
    assert all(v % 1000 < 100 for v in enc.values()), "section would need its first digit"
    return enc, dec


ENC, DEC = build_table()

# ============================================================ codec (numbers)
def encode_block(b4):
    """4 bytes -> 'X4X3X2X1.0YY0YY0YY0YY'"""
    v = [ENC[b] for b in b4]
    return "".join(str(x // 1000) for x in reversed(v)) + "." + "".join(f"{x % 1000:03d}" for x in v)


def decode_block(num):
    ip, fp = num.strip().split(".")
    if len(ip) != 4 or len(fp) != 12 or not (ip + fp).isdigit():
        raise ValueError(f"bad number: {num!r}")
    out = []
    for i in range(4):
        v = int(ip[3 - i]) * 1000 + int(fp[3 * i:3 * i + 3])
        if v not in DEC:
            raise ValueError(f"{v / 1000:.3f} is not a value of this system")
        out.append(DEC[v])
    return bytes(out)


def to_numbers(data):
    pad = (-len(data)) % 4
    data = data + b"\x00" * pad
    return [encode_block(data[i:i + 4]) for i in range(0, len(data), 4)], pad

```

```
125
126 # ============================================ transmission (BCD, no zeros)
127 def numbers_to_digits(nums):
128     """drop the fixed zeros: X4X3X2X1 + YY1..YY4 -> 12 digits per number"""
129     out = []
130     for n in nums:
131         ip, fp = n.split(".")
132         out.append(ip + "".join(fp[3 * i + 1:3 * i + 3] for i in range(4)))
133     return "".join(out)
134
135
136 def digits_to_numbers(d):
137     if len(d) % 12:
138         raise ValueError("digit stream is not a multiple of 12")
139     nums = []
140     for k in range(0, len(d), 12):
141         blk = d[k:k + 12]
142         nums.append(blk[:4] + "." + "".join("0" + blk[4 + 2 * i:6 + 2 * i] for i in range(4)))
143     return nums
144
145
146 def bcd_pack(d):
147     if len(d) % 2:
148         d += "F"                              # unused nibble as padding
149     return bytes(int(d[i], 16) << 4 | int(d[i + 1], 16) for i in range(0, len(d), 2))
150
151
152 def bcd_unpack(b, ndigits):
153     s = "".join(f"{x:02X}" for x in b)[:ndigits]
154     if s and not s.isdigit():
155         raise ValueError("invalid BCD digit in payload")
156     return s
157
158
159 # ================================================================== frame
160 MAGIC = b"PSE2"
161 MODE_BCD, MODE_RAW = 1, 2
162 FLAG_ENC = 0x01
163 HDR = struct.Struct(">4sBBIII")            # magic, mode, flags, original length, body length, crc32
164
165
166 def make_frame(data, mode="bcd"):
167     if mode == "raw":
168         payload, m = data, MODE_RAW
169     else:
170         nums, _ = to_numbers(data)
171         payload, m = bcd_pack(numbers_to_digits(nums)), MODE_BCD
172     crc = zlib.crc32(data) & 0xFFFFFFFF
173     hdr = HDR.pack(MAGIC, m, 0, len(data), len(payload), crc)
174     body = payload
175     return hdr + body
176
177
178 def parse_frame(frame):
179     if len(frame) < HDR.size:
180         raise ValueError("frame too short")
181     magic, mode, flags, n, blen, crc = HDR.unpack_from(frame)
182     if magic != MAGIC:
183         raise ValueError("not a PSE frame")
184     hdr, body = frame[:HDR.size], frame[HDR.size:HDR.size + blen]
185     if len(body) != blen:
186         raise ValueError("truncated frame")
187     if flags & FLAG_ENC:
188         raise ValueError("frame is encrypted: use pse_net_secure.py")
189     payload = body
```

```
    if mode == MODE_RAW:
        data = payload
        nums, _ = to_numbers(data)            # receiver rebuilds the system's numbers
    elif mode == MODE_BCD:
        blocks = (n + 3) // 4
        nums = digits_to_numbers(bcd_unpack(payload, blocks * 12))
        data = b"".join(decode_block(x) for x in nums)[:n]
    else:
        raise ValueError(f"unknown mode {mode}")
    if zlib.crc32(data) & 0xFFFFFFFF != crc:
        raise ValueError("CRC-32 mismatch: data corrupted in transit")
    return data, nums


# ============================================================== network
def recv_exact(conn, n):
    buf = bytearray()
    while len(buf) < n:
        chunk = conn.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("connection closed early")
        buf += chunk
    return bytes(buf)


def send_data(host, port, data, mode="bcd"):
    frame = make_frame(data, mode)
    with socket.create_connection((host, port), timeout=10) as s:
        s.sendall(struct.pack(">I", len(frame)) + frame)
        ack = recv_exact(s, 2)
    return len(frame), ack == b"OK"


def handle(conn):
    size = struct.unpack(">I", recv_exact(conn, 4))[0]
    if size > 2 ** 31:
        conn.sendall(b"ER")
        return None, None, "frame too large"
    frame = recv_exact(conn, size)
    try:
        data, nums = parse_frame(frame)
        conn.sendall(b"OK")
        return data, nums, None
    except ValueError as e:
        conn.sendall(b"ER")
        return None, None, str(e)


def serve(port, out_path=None, once=False, ready=None, result=None):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("0.0.0.0", port))
        srv.listen()
        srv.settimeout(1.0)                   # lets Ctrl+C work on Windows as well
        if ready:
            ready.set()
        while True:
            try:
                conn, addr = srv.accept()
            except socket.timeout:
                continue
            conn.settimeout(30)
            with conn:
                try:
                    data, nums, err = handle(conn)
```

```
                except (OSError, ConnectionError, struct.error) as e:
                    data, nums, err = None, None, f"connection error: {e}"   # server keeps running
            if err:
                print(f"[{addr[0]}] rejected: {err}")
            else:
                print(f"[{addr[0]}] {len(data)} bytes received and verified ({len(nums)} numbers)")
                if out_path:
                    with open(out_path, "wb") as f:
                        f.write(data)
                if result is not None:
                    result.append((data, nums))
            if once:
                return


# ============================================================== tests
def selftest():
    import random
    vals = list(ENC.values())
    assert len(set(vals)) == 256
    print(f"table          : 256 distinct values, largest {max(vals) / 1000:.3f}, every section starts with
0")
    assert all(DEC[ENC[b]] == b for b in range(256))
    ex = bytes([0b11101000, 0b11111000, 0b11101011, 0b10011010])
    n = encode_block(ex)
    print(f"example        : {ex.hex()} -> {n} -> transmitted digits {numbers_to_digits([n])}")
    assert decode_block(n) == ex
    rnd = random.Random(7)
    for size in (0, 1, 3, 4, 5, 1000):
        d = bytes(rnd.randrange(256) for _ in range(size))
        for mode in ("bcd", "raw"):
            assert parse_frame(make_frame(d, mode))[0] == d
    bad = bytearray(make_frame(ex * 10, "bcd")); bad[-1] ^= 0x01
    try:
        parse_frame(bytes(bad)); raise AssertionError("corruption not detected")
    except ValueError:
        pass
    sw = bytearray(make_frame(bytes([1, 2, 3, 4]), "raw")); sw[-4], sw[-3] = sw[-3], sw[-4]
    try:
        parse_frame(bytes(sw)); raise AssertionError("swap not detected")
    except ValueError:
        pass
    print("frames         : bcd and raw round trips OK, corruption and byte swap detected (CRC-32)")
    d = os.urandom(100_000)
    sizes = {m: len(make_frame(d, m)) for m in ("bcd", "raw")}
    print(f"sizes (100 KB) : bcd {sizes['bcd']:,} bytes (x{sizes['bcd'] / len(d):.2f}), "
          f"raw {sizes['raw']:,} bytes (x{sizes['raw'] / len(d):.2f})")
    ready, result = threading.Event(), []
    t = threading.Thread(target=serve, kwargs=dict(port=50555, once=True, ready=ready, result=result),
daemon=True)
    t.start(); ready.wait(5)
    sent, ok = send_data("127.0.0.1", 50555, d, "bcd")
    t.join(5)
    assert ok and result and result[0][0] == d
    print(f"network        : {len(d):,} bytes sent over TCP as {sent:,} bytes, received and verified")


# ================================================================== CLI
def main():
    p = argparse.ArgumentParser(description="Weighted Pair-Symbol Encoding over the network")
    sub = p.add_subparsers(dest="cmd", required=True)
    e = sub.add_parser("encode"); e.add_argument("file")
    d = sub.add_parser("decode"); d.add_argument("numbers", nargs="+")
    r = sub.add_parser("receive"); r.add_argument("--port", type=int, default=5050)
```

```
    r.add_argument("--out"); r.add_argument("--once", action="store_true")
    s = sub.add_parser("send"); s.add_argument("file"); s.add_argument("--host", default="127.0.0.1")
    s.add_argument("--port", type=int, default=5050); s.add_argument("--mode", choices=["bcd", "raw"],
default="bcd")
    sub.add_parser("selftest")
    a = p.parse_args()
    try:
        if a.cmd == "encode":
            nums, pad = to_numbers(open(a.file, "rb").read())
            print("\n".join(nums))
            if pad:
                print(f"# last block padded with {pad} zero byte(s)", file=sys.stderr)
        elif a.cmd == "decode":
            print(b"".join(decode_block(n) for n in a.numbers).hex())
        elif a.cmd == "receive":
            print(f"listening on port {a.port} ... (Ctrl+C to stop)")
            try:
                serve(a.port, a.out, a.once)
            except KeyboardInterrupt:
                print("stopped")
        elif a.cmd == "send":
            data = open(a.file, "rb").read()
            size, ok = send_data(a.host, a.port, data, a.mode)
            print(f"sent {len(data):,} bytes as {size:,} bytes ({a.mode}) -> {'verified' if ok else
'REJECTED'}")
        elif a.cmd == "selftest":
            selftest()
    except (ValueError, OSError, ConnectionError) as ex:
        print(f"error: {ex}", file=sys.stderr)
        sys.exit(1)


if __name__ == "__main__":
    main()
```